

THE UNIVERSITY OF DAR ES SALAAM



COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

COURSE: IS 365 - PRACTICAL ASSIGNMENT (LOCAL LLM APPLICATION)

TASK 9 - INDUSTRY AND

PRODUCTION REFLECTION

IS 365 DEPLOYMENT AND SECURITY ANALYSIS

STUDENT'S DETAILS:

NAME	REGISTRATION NUMBER	PROGRAMME
Charles J Tungaraza	2023-04-13490	BSc Computer Science
Stella Thomas Kahungo	2023-04-03737	BSc Computer Science
Godbless Kaaya	2023-04-03579	BSc Computer Science
Baraka Jimmy Maengesho	2023-04-06473	BSc Computer Science
Frank Elikana Wallace	2023-04-13642	BSc Computer Science
Kelvin George Msuya	2023-04-08929	BSc BIT

1. What are the main components of your deployed LLM system?

On our laptop the system has six parts:

1. **Streamlit frontend** - the chat page (port 8501)
2. **FastAPI backend** - the API in the middle (port 8000)
3. **Ollama** - runs the llama3.2:1bmodel (port 11434)
4. **UDSM FAQ file** - our markdown notes used for RAG
5. **Log file** - backend/logs/app.log
6. **Feedback file** - data/feedback.jsonl for Good/Average/Poor ratings

“Deployed” here means **running locally** as a class prototype, not on a public cloud server.

2. Why is FastAPI useful in this pipeline?

FastAPI gave us proper HTTP endpoints (/health, /ask, /feedback) and Swagger at /docs for testing. Pydantic blocks empty questions (422). Most importantly, the **frontend never talks to Ollama directly** - everything goes through the API. That is how many real projects are structured: UI → API → model.

3. What role does your chosen LLM model play?

llama3.2:1b is a small open model. Its job is to **turn the question + FAQ context + prompt into readable text**. It does not search the FAQ - our backend does that first. We picked this model because it fits 8 GB RAM. It runs locally, so we do not pay per message like a cloud API.

4. What role does the frontend play?

Streamlit is what a student actually uses: type a question, wait, read the answer, maybe rate it. It also shows errors clearly (for example when the backend is off) and has sidebar switches for RAG, history, and streaming. Without a frontend, users would need Swagger or curl, which is fine for developers but not for students.

5. What is the difference between running the model locally and using an external API?

Local (what we did): Data stays on the machine after setup. No API key. Free after download. But slower on a normal laptop CPU and the model is weaker.

External API (example. ChatGPT): Usually faster and smarter, but you need internet, an account, payment, and your questions leave your computer.

We used local because the assignment required it and we wanted to understand the full pipeline ourselves.

6. What security risks may exist if this system is deployed in an organisation?

If UDSM used our prototype as-is, risks include:

- No login - anyone with the link could use it
- No HTTPS on localhost (fine for demo, not for campus network)
- Student questions stored in plain log files
- No rate limiting
- The model can still give wrong answers (hallucination)
- Our FAQ is demo text, not official policy

These are not acceptable for a real production service.

7. What improvements would be needed before deploying this system in production?

We would need HTTPS, student authentication (perhaps linked to ARIS), an official UDSM knowledge base maintained by staff, a stronger server or GPU, monitoring and alerts, rate limits, privacy rules for logs, and a way to escalate to a human when the bot is unsure.

8. How would you monitor the system in real-world use?

We would ping /health regularly, send logs to a central system, track how long /ask takes, count 503/504 errors, and review Poor ratings in the feedback file. During busy times (registration week) uptime would matter more.

9. How would you protect sensitive student information?

Use HTTPS everywhere, require login, avoid saving unnecessary personal data in logs, limit how long logs are kept, restrict who can read them, and show students a short privacy notice. We would follow UDSM ICT policies and national data protection rules.

10. What challenges did you face during implementation?

The main ones were **limited RAM** (8 GB), **slow answers on CPU**, the **size of the Ollama installer**, writing **demo FAQ content**, and small **Windows/PowerShell** differences from Linux tutorials. We also had to restart services several times when port 8000 was already in use. Overall the project was doable on our hardware but needed patience during demos.

Submitted with the IS 365 technical report (submit_report.pdf).

Related documents

Document	Use
submit_report.md	Technical report
architecture.md	System components

learning_outcomes.md	Outcome 8 - prototype vs production
error_handling.md	Security and error handling
README.md	Full docs index